

What has to happen before this app is deployed

A complete account of the current state: what is verified working, what is broken, what is untested, and the exact steps remaining. Written to be read by someone who was not in the room.

Prepared 11 Aug 2026 Backend live · verified App unverified Blockers 3

THE ONE-PARAGRAPH VERSION

Three days ago the app was completely dead in production. The database had switched itself off and its address stopped existing, so every request failed. That is fixed: the app now runs on the client's own Supabase, and nine separate security holes were closed and individually verified against the live database.

But the app itself has still never been opened by a human since any of this changed.

Everything verified so far was verified by calling the server directly. The screens, the camera, the photo picker and real payments remain unproven. That is the single largest unknown, and it is the next thing that should happen.

1 · WHAT IS ACTUALLY WORKING

Each of these was tested against the live production server, not assumed from reading the code. Where something is marked verified, there is a real request and a real response behind it.

AREA	HOW IT WAS PROVEN
Account creation & login	Real account created, logged in, session used to call protected routes
Correct role on signup	Created a coach; confirmed it stayed a coach rather than silently becoming an athlete
Main screens' data	Discover, profile, subscriptions, rankings, posts, messages, matches, identity check — all returned 200
Photo & video upload	Real JPEG uploaded through a signed URL and stored
Malicious file blocking	An HTML file upload was <i>refused</i> — this attack is closed
Live chat connection	Authenticated socket connected; an unauthenticated one was refused
Guardian consent flow	Athlete QR → parent scan → link created → consent video uploaded to the private area
Phone signup & login	Full loop using a test number, no SMS needed
Draft allowance	Reports 20 per <i>month</i> , correcting a bug that reset it daily
Public key cannot read data	The key inside the app is now refused by the database

BLOCKER Nobody has opened the app

Every check above was made by talking to the server directly. Not one of them proves a button works, a camera opens, or a screen renders. The app has not been launched by a person since the database moved, the storage rules changed, and the SMS provider was replaced.

Why this matters more than it sounds

A server can answer perfectly while the app fails to display the answer. The parts most likely to break are exactly the parts a server test cannot reach: the photo picker, the camera, the QR scanner, and anything that renders an image from storage — because the storage rules were rewritten.

Exactly what to do

1. Wait for the build to finish, then install it on a real phone.
2. Create a brand-new account. Old accounts no longer exist — the database started empty.
3. Complete onboarding all the way through.
4. Upload a profile photo. *Most likely thing to fail.*
5. Upload a video. Note the new limit is 15 MB, reduced from 50 MB.
6. Open Discover and swipe.
7. Send a message to someone.
8. Attempt a purchase.

Write down anything that looks wrong, even cosmetic. Nothing on this list has been seen working.

Owner Achraf

Blocked by build finishing

Effort ~30 minutes

BLOCKER

No SMS can reach a real phone

The app moved from Twilio to Prelude. The integration is finished, deployed and confirmed working — but the Prelude account has a balance of **€0.00**. Any attempt to text a real phone is refused with `insufficient_balance`.

What this breaks

- Signing up with a phone number
- Logging in with a phone number
- Any user without an email address is locked out entirely

What still works without paying

One test number is registered and functions with no credit, because no real message is sent:

Phone: +213558780131
Code: 123456

This is enough for testing and for the store reviewer, but no real user can receive a code.

Exactly what to do

1. Open `app.prelude.so`, signed in as the client.
2. Press **Top Up** (top right — it already says "Top Up Required").
3. Add credit. A small amount covers a great many messages.
4. Tell me, and I will send a real code to a real phone to confirm.

Owner Patrick

Cost small — roughly €20 goes a long way

Effort 5 minutes

BLOCKER Store listing paperwork is not started

The build is only one part of publishing. Google will not accept the app without several forms completed, and the most common rejection for an app like this is a reviewer who cannot get past the login screen.

The reviewer trap, in detail

GetDraft shows nothing at all until you are signed in. A reviewer who cannot sign in marks the app as broken and rejects it. There is a field in the console specifically for giving them an account, and it must be filled in.

Use the test number above. And add this sentence, which prevents the second most likely rejection:

When asked for a date of birth, please enter an adult date. Accounts under 18 are held for guardian approval by design.

Without that note, a reviewer may enter a teenage birthday, land in the guardian-approval waiting state, conclude the app is stuck, and reject it.

Everything the console needs

- The app created under the identifier `com.getdraft.app`
- **App access** — the login above plus the date-of-birth note
- **Store listing** — icon, a 1024×500 banner, 2–8 screenshots, a short description (80 characters) and a long one
- **Data safety** — an honest declaration of what the app collects
- **Content rating** — a questionnaire; declare messaging and user-posted content truthfully
- **Target audience** — this app is used by minors, which triggers extra review
- **Privacy policy link** — already live and working

Owner Achraf, with the client's account

Effort 1–2 hours

Can start now — does not need the build

RISK

Photos and videos of minors are readable by anyone with the link

Five of the six storage areas are public. Public here has a specific and strong meaning: any person holding the web address of a file can open it. No login. No expiry. And once a link escapes, it cannot be revoked.

Two of those areas hold photos and videos of athletes, who are frequently under 18.

The consent recordings are correctly private — that part was already right.

Why it is not marked a blocker

Because it is a product decision, not a defect. A recruiting app is meant to be seen. But *"visible to signed-in recruiters"* and *"visible to the entire internet"* are different promises, and right now it is the second one. This also has to match whatever is declared on the data safety form.

The fix, and its cost

Switching them to private is a single line of configuration. The app then has to request temporary links for every image instead of using permanent ones — a real change, best done alongside other app work rather than alone.

Decision needed from Patrick

RISK**In-app purchases may cause store rejection**

The app charges for subscriptions and Draft packs using Stripe's own payment sheet, inside the app. Both Google and Apple require purchases of digital goods to go through their own billing systems, and take a percentage.

This was temporarily removed and then restored at the owner's instruction, which is a legitimate call — the rules have been shifting following recent court decisions, and it may now be permitted in some countries.

The two possible outcomes

- **Rejected at review.** Annoying, quick to recover from.
- **Accepted, then removed later.** Considerably worse, because by then there are real users.

If it is rejected

The switch to turn purchases off is already written and tested. It is one line in `constants/purchases.ts`. Users would then subscribe through the website instead, and the app would simply reflect their plan. Nothing else needs to change.

Accepted knowingly by the owner

RISK The database can switch itself off again

This is the exact failure that caused the outage. The free plan suspends a project after roughly seven days without activity, and when it suspends, its web address stops existing. The app then fails completely.

It is hard to notice, because the health check returns "healthy" even when the database behind it is gone. That is why it went undiagnosed.

The fix is written but switched off

An automatic ping every ten minutes is already committed to the repository. It does nothing until two values are added to GitHub — and neither of them is actually secret, since both already ship inside the app where anyone can read them.

Repository → Settings → Secrets and variables → Actions

```
SUPABASE_URL      https://icczjnsevyczfllsiamu.supabase.co
SUPABASE_ANON_KEY (the "anon" value in .secrets/client_keys.json)
```

Then run it once by hand from the Actions tab to confirm it passes.

Owner Achraf

Effort 2 minutes

RISK There are no backups

The free plan includes none. Not reduced backups — none. If data is deleted or corrupted, it is gone permanently.

This is currently fine, and deliberately deferred: the database holds two test accounts and no files. There is nothing to lose.

The moment that stops being true

The first real athlete who uploads a highlight reel and pays for a subscription. That data cannot be recreated. A nightly backup is already written and committed; enabling it later is one extra value in GitHub.

Trigger first real user data — not launch day

RISK Storage runs out after roughly 65 videos

The free plan allows 1 GB in total, across every user, permanently. The original limits allowed 50 MB per video, which would have filled the entire project after about twenty clips. Limits were reduced to 15 MB, giving roughly sixty-five.

That is enough for testing with a dozen people. It is not enough for a public launch of a video-based product, and no amount of adjustment makes 1 GB sufficient.

When it fills, uploads simply start failing. There is no warning and no grace period.

Options paid plan (100 GB), or move video to a separate provider

RISK Four passwords and keys were shared in chat

During setup, four live credentials were typed into a conversation. All four still work. All four can be replaced in a few clicks, and none of them requires a code change, because the app reads them from configuration.

- The database password
- A Supabase administrative token
- The email-sending key
- The SMS provider key

Not urgent while the app has no real users. Worth doing before it does.

NOT STARTED iPhone

The iPhone version **has never been built. Not once.** It has never been compiled, never installed, never opened. Nobody knows whether it starts.

Everything in this document concerns Android. iPhone is a separate piece of work, and the honest estimate is weeks rather than days — not because it is difficult, but because nothing about it has ever been verified.

What it additionally needs

- An Apple developer account — \$99 per year, with identity checks that take days
- Certificates for notifications
- A privacy declaration file Apple now requires
- A first build, then a first test on a real device

Apple also enforces the purchase rules described earlier more strictly than Google does.

Recommendation: publish Android, learn from it, then start iPhone.

#	STEP	WHO	TIME
1	Add the two values to GitHub, run the keep-alive once	Achraf	2 min
2	Complete the store listing paperwork	Achraf	1–2 h
3	Add credit to the SMS account	Patrick	5 min
4	Install the finished build and use every screen	Achraf	30 min
5	Fix whatever step 4 uncovers	Achraf	unknown
6	Upload to internal testing, invite testers	Achraf	20 min
7	Decide the photo and video privacy question	Patrick	–
8	Submit for review	Achraf	30 min

Steps 1 to 3 can all happen today and do not depend on each other. Step 4 is the one that decides whether the schedule is real, because it is the first time anyone will have seen the app working. Treat any estimate past that point as provisional.

6 · WHAT WAS FIXED, FOR THE RECORD

Nine security problems were closed. Every one was reachable using the key that ships inside the app, which anybody can extract in a few minutes. Each fix was confirmed against the live database.

PROBLEM	WHAT IT ALLOWED
Self-promotion to administrator	Any user could make themselves an admin
Administrator signup	Registering while requesting the admin role created one
Personal data exposure	Every user's email, phone and payment identifiers were readable
Paywall bypass	Unlimited Drafts by writing to the database directly
Message forgery	Sending a message that appeared to come from the other person
Ranking manipulation	An athlete could set their own view and like counts
Age-check bypass	An athlete could change their own date of birth
Fake verification badge	A recruiter could mark themselves verified
Two data leaks by identifier	Private profile and ranking data exposed to anyone who guessed an ID

Also fixed

- **A revenue leak.** The free Draft allowance reset every day instead of every month, making the free tier roughly thirty times more generous than intended — for precisely the users who would otherwise have paid.
- **Lost purchases.** Two Draft packs bought at the same moment could overwrite each other, so a customer paid and received nothing.
- **A login bypass.** One real phone number could sign in using the code `000000`, with no message ever sent. It was live in production.
- **Uploads accepted any file type.** Including web pages, which can run code in a visitor's browser when served from your own domain.
- **Tests that proved nothing.** Part of the test suite passed while silently skipping the code it claimed to check. Rewritten, then confirmed by deliberately reintroducing a bug to make sure the

tests actually failed.

7 · REFERENCE

Test login (works today, no credit required)

Phone: +213558780131
Code: 123456

Delete this test number before the app is public. While it exists, anyone who knows that number can sign in as it. It grants no special access and reaches nobody else's data, but it should not outlive testing.

Live addresses

API	https://getdraft-api-production.up.railway.app/api
Privacy	https://getdraft-api-production.up.railway.app/api/privacy
Terms	https://getdraft-api-production.up.railway.app/api/terms
Database	Supabase project icczjnsevyczflsiamu (client-owned)

Things a server test cannot prove, and still have not been proven

- Any screen in the app renders correctly
- The camera and photo picker work
- A notification arrives on a real phone
- A real payment completes and unlocks the right plan
- The iPhone version starts at all